in post-processing, it helps adjust the predictions after training.

Let us now look at a coding example that will help you understand this better. You can open any code editor you like — I will be using Google Colab. You can open a Jupyter Notebook through the terminal, or on VSCode, or anywhere that you feel comfortable.

Open Google Colab using the link *https://colab.research.google.com.*

Once your Jupyter Notebook is up and running, you can install Fairlearn using the following command:

```
pip install fairlearn
```

We will try this example with a sample random dataset. For this, import all the required libraries in a new cell as shown below:

```
import numpy as np
import pandas as pd
from sklearn.linear_model import
LogisticRegression
from sklearn.model_selection import
train_test_split
from sklearn.metrics import accuracy_
score, confusion_matrix
from fairlearn.metrics import
MetricFrame, false_positive_rate
from fairlearn.reductions import
ExponentiatedGradient, DemographicParity
```

We are importing NumPy and Pandas to generate and use the dataset, and importing logistic regression from sklearn to build our model. We first fix the random seed so that the results are reproducible. We will then generate a simple feature set with two numeric features and 100 rows. This is purely synthetic data, just as an example for us to use in this code. Next, we create a binary target variable simulating a label. We can do this using the following code:

```
np.random.seed(42)
X = pd.DataFrame({
   "feature1": np.random.randn(100),
   "feature2": np.random.randn(100)
})
y = np.random.choice([0, 1], size=100)
```

The following code represents sensitive attributes, i.e., the columns that we'll use to evaluate fairness. They would usually be features like gender, race, etc, that may cause bias and unfairness.

```
sensitive_feature = np.random.
choice(["GroupA", "GroupB"], size=100)
```

We then split the features, labels, and sensitive attributes into training and test sets, keeping 30% of the data aside for evaluation using the following code:

```
X_train, X_test, y_train, y_test, s_
train, s_test = train_test_split(
   X, y, sensitive_feature, test_
size=0.3
)
```

We start with a plain logistic regression model. At this point, there is nothing fairness-aware about it -- it's just optimising for prediction accuracy. The model has been trained using the training features and labels, learning a set of coefficients that best separate the two classes.

```
clf = LogisticRegression()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
```

The next step is where fairness comes into play. MetricFrame calculates metrics separately for each sensitive group, allowing us to see whether performance differs between GroupA and GroupB. In addition to accuracy, we include false positive rate, which is often critical in fairness discussions. We then get the metrics as shown in Figure 1.

```
Baseline model metrics by group:
                    accuracy  false_positive_rate
sensitive_feature_0
GroupA              0.428571                  1.0
GroupB              0.500000                  1.0
```

Figure 1: Metrics by groups

```
metrics = MetricFrame(
   metrics={"accuracy": accuracy_score,
"false_positive_rate": false_positive_
rate},
   y_true=y_test,
   y_pred=y_pred,
   sensitive_features=s_test
)
print("Baseline model metrics by
group:")
print(metrics.by_group)
```

Next, we set up a fairness-aware version of logistic regression using Fairlearn's Exponentiated Gradient algorithm. By specifying demographic parity as the constraint, we're asking the model to balance positive prediction rates across groups. This can be done using the following code:

```
mitigator = ExponentiatedGradient(
   estimator=LogisticRegression(),
   constraints=DemographicParity()
)
mitigator.fit(X_train, y_train,
sensitive_features=s_train)
y_pred_mitigated = mitigator.predict(X_
test)
```

The model is then retrained with access to the sensitive feature, allowing it to explicitly account for group membership while optimising under the fairness constraint. We then generate predictions from this mitigated model to see how its behaviour compares with the baseline. We can do this using the following code and the results would look like what's shown in Figure 2.